

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Artificial Intelligence Application Development and Jira

PRACTICAL VIII

Roll No.: T005	Name: Ritesh Chaubey
Class: TYIT	Batch: 1
Date of Assignment: 22-08-2026	Date/Time of Submission: 22-08-2026

Aim:-Ethics and Bias Analysis

Select any publicly available dataset (for example, Iris, Adult Income, or a classification dataset of your choice) and analyze it for possible bias and class imbalance.

Code:-

```
1)Titanic train.csv from Kaggle
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_style("whitegrid")
# -----
# 1. Load and Explore the Dataset
# -----
df = pd.read_csv("train.csv")
print("\n===== 1. DATASET INFORMATION =====")
print("First 5 rows:")
print(df.head())
print("\nDataset Shape:")
print(df.shape)
print("\nColumn Names:")
print(df.columns.tolist())
print("\nDataset Information:")
print(df.info())
print("\nStatistical Summary:")
print(df.describe())
# -----
# 2. Check Data Quality
# -----
print("\n===== 2. DATA QUALITY =====")
print("\nMissing Values:")
print(df.isnull().sum())
print("\nDuplicate Rows:")
print(df.duplicated().sum())
print("\nData Types:")
print(df.dtypes)
# -----
# 3. Identify the Target Variable
# -----
```

```

target = "Survived"
print("\n===== 3. TARGET VARIABLE =====")
print("Target Variable:", target)
print("Target Values:")
print(df[target].value_counts())
# -----
# 4. Analyze Class Distribution
# -----
print("\n===== 4. CLASS DISTRIBUTION =====")
class_distribution = df[target].value_counts()
print(class_distribution)
print("\nClass Distribution Percentage:")
print(df[target].value_counts(normalize=True) * 100)
# -----
# 5. Visualize Class Imbalance
# -----
plt.figure(figsize=(7, 5))
sns.countplot(data=df, x="Survived", hue="Survived", palette="Set2")
plt.title("Titanic Survival Class Distribution")
plt.xlabel("Survived (0 = No, 1 = Yes)")
plt.ylabel("Number of Passengers")
plt.tight_layout()
plt.show()
# -----
# 6. Analyze Gender Representation and Display Graph
# -----
print("\n===== 6. GENDER REPRESENTATION =====")
print(df["Sex"].value_counts())
plt.figure(figsize=(7, 5))
sns.countplot(data=df, x="Sex", hue="Sex", palette="pastel")
plt.title("Gender Representation")
plt.xlabel("Gender")
plt.ylabel("Number of Passengers")
plt.tight_layout()
plt.show()
# -----
# 7. Analyze Survival by Gender and Display Graph
# -----
print("\n===== 7. SURVIVAL BY GENDER =====")
gender_survival = pd.crosstab(
    df["Sex"],
    df["Survived"],
    normalize="index"
) * 100
print(gender_survival)
plt.figure(figsize=(8, 5))
sns.countplot(
    data=df,
    x="Sex",
    hue="Survived",
    palette="Set1"

```

```

)
plt.title("Survival by Gender")
plt.xlabel("Gender")
plt.ylabel("Number of Passengers")
plt.legend(title="Survived", labels=["No", "Yes"])
plt.tight_layout()
plt.show()
# -----
# 8. Analyze Passenger-Class Representation
# -----
print("\n===== 8. PASSENGER CLASS REPRESENTATION =====")
print(df["Pclass"].value_counts().sort_index())
plt.figure(figsize=(7, 5))
sns.countplot(
    data=df,
    x="Pclass",
    hue="Pclass",
    palette="Blues"
)
plt.title("Passenger-Class Representation")
plt.xlabel("Passenger Class")
plt.ylabel("Number of Passengers")
plt.tight_layout()
plt.show()
# -----
# 9. Analyze Survival by Passenger Class
# -----
print("\n===== 9. SURVIVAL BY PASSENGER CLASS =====")
class_survival = pd.crosstab(
    df["Pclass"],
    df["Survived"],
    normalize="index"
) * 100
print(class_survival)
plt.figure(figsize=(8, 5))
sns.countplot(
    data=df,
    x="Pclass",
    hue="Survived",
    palette="Set2"
)
plt.title("Survival by Passenger Class")
plt.xlabel("Passenger Class")
plt.ylabel("Number of Passengers")
plt.legend(title="Survived", labels=["No", "Yes"])
plt.tight_layout()
plt.show()
# -----
# 10. Analyze Age Groups
# -----
print("\n===== 10. AGE GROUPS =====")

```

```

# Create age groups
df["AgeGroup"] = pd.cut(
    df["Age"],
    bins=[0, 12, 18, 35, 60, 100],
    labels=["Child", "Teenager", "Young Adult", "Adult", "Senior"]
)
print(df["AgeGroup"].value_counts())
plt.figure(figsize=(8, 5))
sns.countplot(
    data=df,
    x="AgeGroup",
    hue="AgeGroup",
    order=["Child", "Teenager", "Young Adult", "Adult", "Senior"],
    palette="viridis"
)
plt.title("Age Group Representation")
plt.xlabel("Age Group")
plt.ylabel("Number of Passengers")
plt.xticks(rotation=20)
plt.tight_layout()
plt.show()
# -----
# 11. Analyze Survival by Age Group
# -----
print("\nn===== 11. SURVIVAL BY AGE GROUP =====")
age_survival = pd.crosstab(
    df["AgeGroup"],
    df["Survived"],
    normalize="index"
) * 100
print(age_survival)
plt.figure(figsize=(9, 5))
sns.countplot(
    data=df,
    x="AgeGroup",
    hue="Survived",
    order=["Child", "Teenager", "Young Adult", "Adult", "Senior"],
    palette="Set1"
)
plt.title("Survival by Age Group")
plt.xlabel("Age Group")
plt.ylabel("Number of Passengers")
plt.legend(title="Survived", labels=["No", "Yes"])
plt.xticks(rotation=20)
plt.tight_layout()
plt.show()
Output:-

```

===== 1. DATASET INFORMATION =====

First 5 rows:

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	
4	5	0	3	

	Name	Sex	Age	SibSp	\
0	Braund, Mr. Owen Harris	male	22.0	1	
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	
2	Heikkinen, Miss. Laina	female	26.0	0	
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	
4	Allen, Mr. William Henry	male	35.0	0	

	Parch	Ticket	Fare	Cabin	Embarked
0	0	A/5 21171	7.2500	NaN	S
1	0	PC 17599	71.2833	C85	C
2	0	STON/O2. 3101282	7.9250	NaN	S
3	0	113803	53.1000	C123	S
4	0	373450	8.0500	NaN	S

Dataset Shape:
(891, 12)

Dataset Shape:
(891, 12)

Column Names:

['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp', 'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked']

Dataset Information:

<class 'pandas.DataFrame'>

RangeIndex: 891 entries, 0 to 890

Data columns (total 12 columns):

#	Column	Non-Null Count	Dtype
0	PassengerId	891 non-null	int64
1	Survived	891 non-null	int64
2	Pclass	891 non-null	int64
3	Name	891 non-null	str
4	Sex	891 non-null	str
5	Age	714 non-null	float64
6	SibSp	891 non-null	int64
7	Parch	891 non-null	int64
8	Ticket	891 non-null	str
9	Fare	891 non-null	float64
10	Cabin	204 non-null	str
11	Embarked	889 non-null	str

dtypes: float64(2), int64(5), str(5)

memory usage: 83.7 KB

None

===== 3. TARGET VARIABLE =====

Target Variable: Survived

Target Values:

Survived

0 549

1 342

Name: count, dtype: int64

Statistical Summary:

	PassengerId	Survived	Pclass	Age	SibSp
count	891.000000	891.000000	891.000000	714.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008
std	257.353842	0.486592	0.836071	14.526497	1.102743
min	1.000000	0.000000	1.000000	0.420000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000
50%	446.000000	0.000000	3.000000	28.000000	0.000000
75%	668.500000	1.000000	3.000000	38.000000	1.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000

	Parch	Fare
count	891.000000	891.000000
mean	0.381594	32.204208
std	0.806057	49.693429
min	0.000000	0.000000
25%	0.000000	7.910400
50%	0.000000	14.454200
75%	0.000000	31.000000
max	6.000000	512.329200

===== 2. DATA QUALITY =====

Missing Values:

PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	177
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	687
Embarked	2

dtype: int64

Duplicate Rows:

0

Data Types:

PassengerId	int64
Survived	int64
Pclass	int64
Name	str
Sex	str
Age	float64
SibSp	int64
Parch	int64
Ticket	str
Fare	float64
Cabin	str
Embarked	str

dtype: object

===== 4. CLASS DISTRIBUTION =====

Survived

0 549

1 342

Name: count, dtype: int64

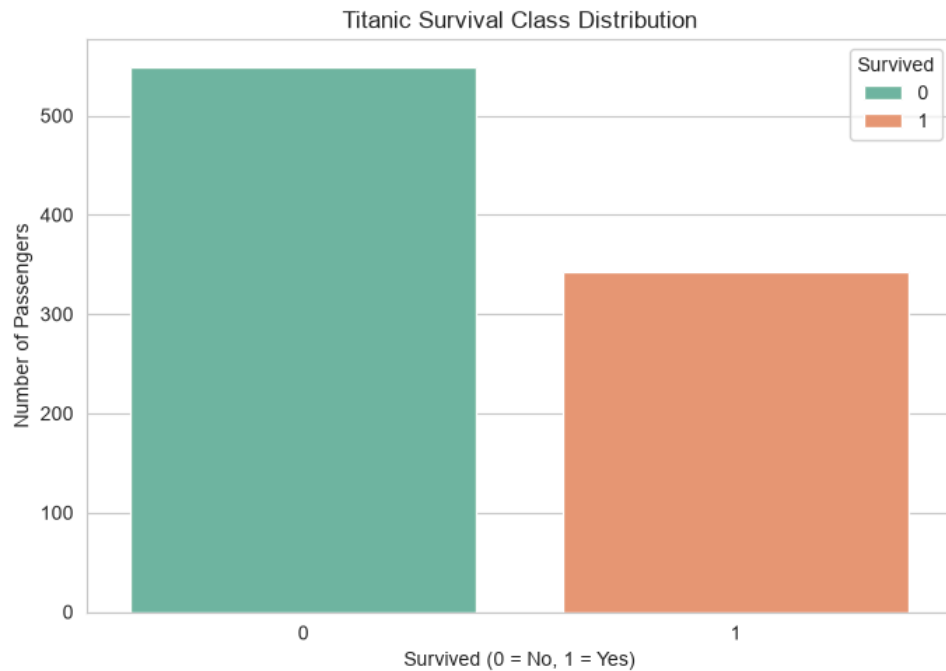
Class Distribution Percentage:

Survived

0 61.616162

1 38.383838

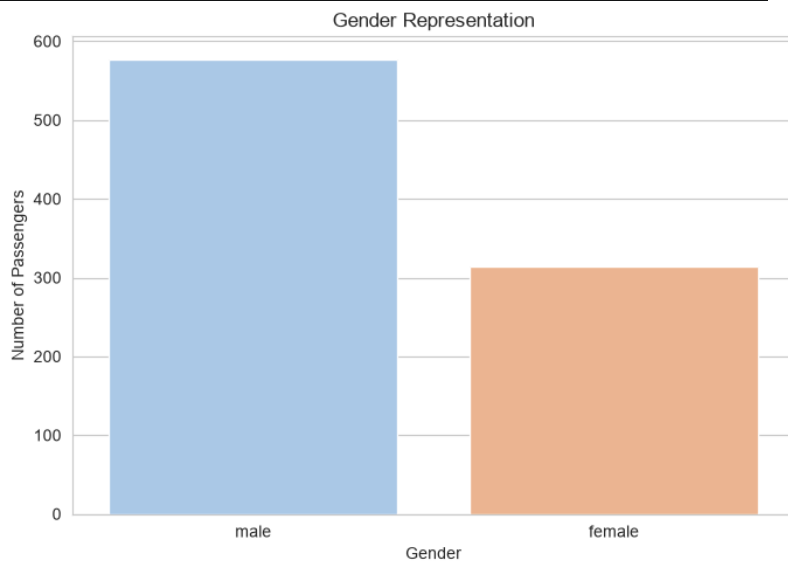
Name: proportion, dtype: float64



```

===== 6. GENDER REPRESENTATION =====
Sex
male      577
female    314
Name: count, dtype: int64

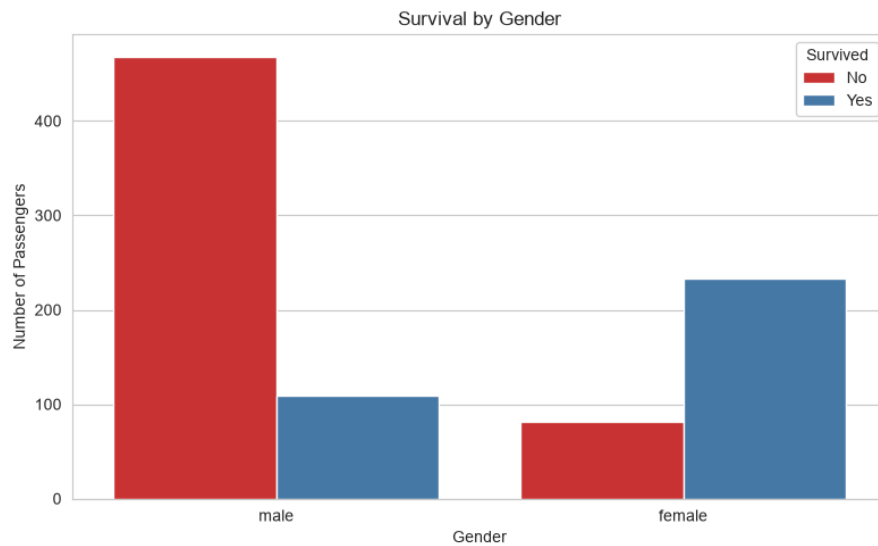
```



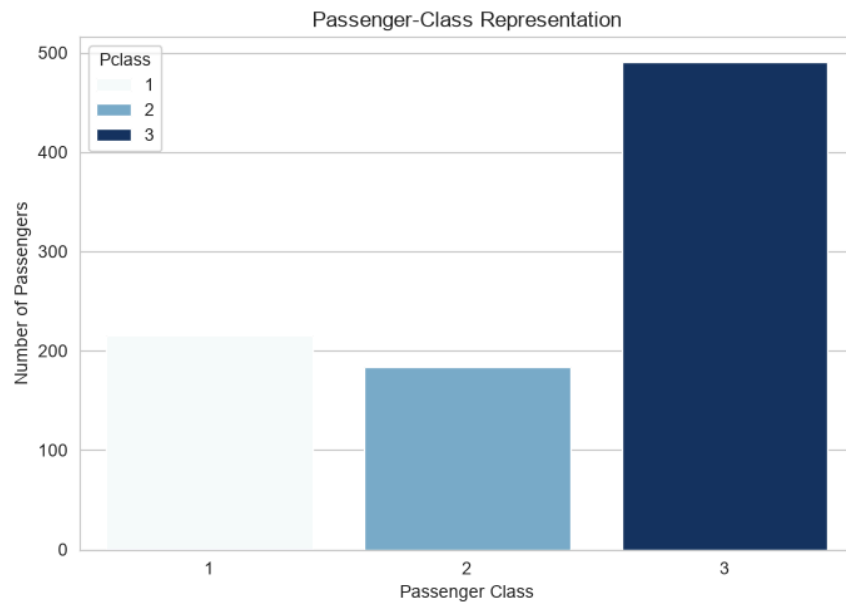
```

===== 7. SURVIVAL BY GENDER =====
Survived      0      1
Sex
female    25.796178  74.203822
male      81.109185  18.890815

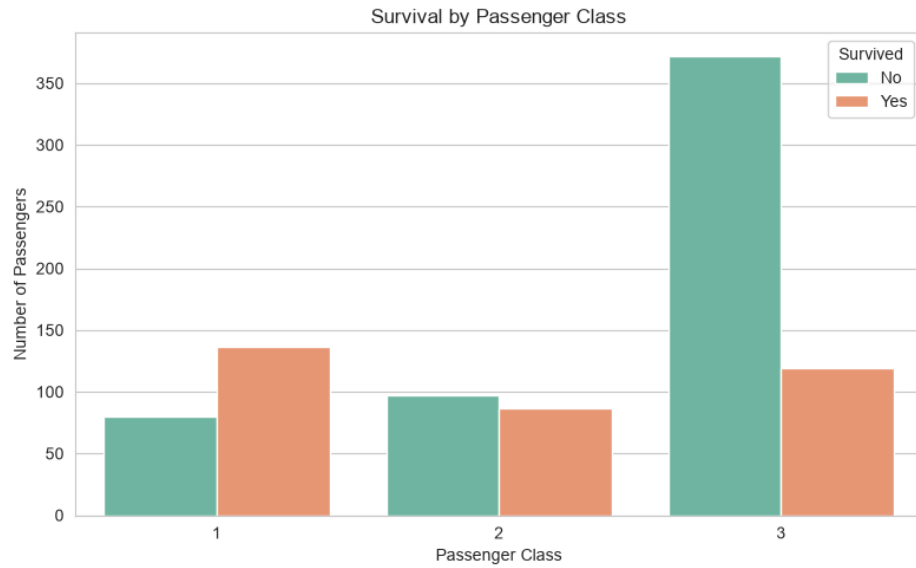
```



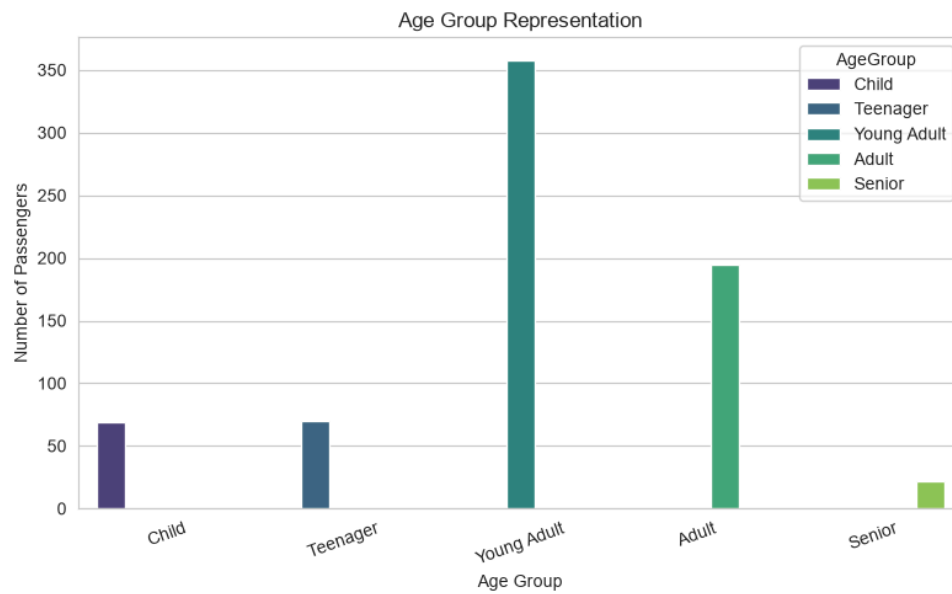
```
===== 8. PASSENGER CLASS REPRESENTATION =====
Pclass
1      216
2      184
3      491
Name: count, dtype: int64
```



```
===== 9. SURVIVAL BY PASSENGER CLASS =====
Survived      0      1
Pclass
1      37.037037  62.962963
2      52.717391  47.282609
3      75.763747  24.236253
```

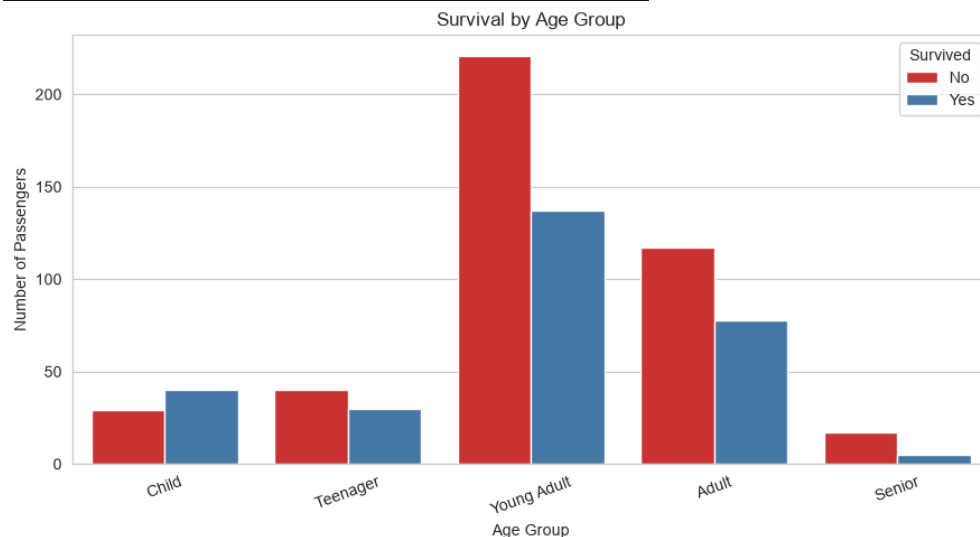


```
===== 10. AGE GROUPS =====
AgeGroup
Young Adult    358
Adult          195
Teenager       70
Child          69
Senior         22
Name: count, dtype: int64
```



```
===== 11. SURVIVAL BY AGE GROUP =====
```

Survived	0	1
AgeGroup		
Child	42.028986	57.971014
Teenager	57.142857	42.857143
Young Adult	61.731844	38.268156
Adult	60.000000	40.000000
Senior	77.272727	22.727273



Titanic Dataset

12. Identify Possible Bias

The Titanic dataset may contain several types of bias. The number of male and female passengers is not equal, and the survival rate also differs significantly between genders. Passenger classes are also represented differently, with different survival rates among the classes. Some age groups have fewer passengers than others, and missing age values may affect the analysis. Since the dataset represents a historical event, social factors such as gender and passenger class strongly influenced survival. Therefore, the dataset contains historical and demographic bias.

13. Impact on Machine Learning

Bias in the Titanic dataset can affect the performance and fairness of a machine learning model. The model may learn strong relationships between gender, passenger class, age and survival instead of learning more general patterns. Class imbalance can cause the model to favor the majority class. Missing values can also reduce prediction accuracy. As a result, the model may perform differently for different demographic groups.

14. Suggested Solutions

The following solutions can be used to reduce the effect of bias:

- Handle missing values using suitable imputation methods.

- Balance the target classes using oversampling, undersampling or SMOTE.

- Use appropriate evaluation metrics such as precision, recall and F1-score instead of relying only on accuracy.

- Analyze model performance separately for different gender, age and passenger-class groups.

- Identify and reduce the effect of features that may introduce unwanted bias.

- Use cross-validation to obtain more reliable model performance.

- Perform fairness analysis before using the model.

15. Final Conclusion

The Titanic dataset shows significant differences in survival based on gender, passenger class and age. These differences can introduce bias into machine learning models. Class imbalance and missing values can further affect model performance. Therefore, proper data preprocessing, class balancing and fairness evaluation are important when developing a machine learning model using this dataset.

B)German Credit Risk Dataset from Kaggle

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_style("whitegrid")
# -----
# 1. Load and Explore the Dataset
# -----
df = pd.read_csv("german_credit_data (1).csv")
print("\n===== 1. DATASET INFORMATION =====")
print("First 5 rows:")
print(df.head())
print("\nDataset Shape:")
print(df.shape)
print("\nColumn Names:")
print(df.columns.tolist())
print("\nDataset Information:")
df.info()
print("\nStatistical Summary:")
print(df.describe(include="all"))
# -----
# 2. Check Data Quality
# -----
print("\n===== 2. DATA QUALITY =====")
print("\nMissing Values:")
print(df.isnull().sum())
print("\nDuplicate Rows:")
print(df.duplicated().sum())
print("\nData Types:")
print(df.dtypes)
# -----
# 3. Identify Target Variable
# -----
target = "Risk"
print("\n===== 3. TARGET VARIABLE =====")
print("Target Variable:", target)
print("\nTarget Values:")
print(df[target].value_counts())
# -----
# 4. Analyze Class Distribution
# -----
```

```

print("\n===== 4. CLASS DISTRIBUTION =====")
print(df[target].value_counts())
print("\nPercentage Distribution:")
print(df[target].value_counts(normalize=True) * 100)
# -----
# 5. Visualize Class Imbalance
# -----
plt.figure(figsize=(7, 5))
sns.countplot(
    data=df,
    x=target,
    hue=target,
    palette="Set2"
)
plt.title("German Credit Risk Class Distribution")
plt.xlabel("Credit Risk")
plt.ylabel("Number of Customers")
plt.tight_layout()
plt.show()
# -----
# 6. Analyze Gender Representation
# -----
print("\n===== 6. GENDER REPRESENTATION =====")
print(df["Sex"].value_counts())
plt.figure(figsize=(7, 5))
sns.countplot(
    data=df,
    x="Sex",
    hue="Sex",
    palette="pastel"
)
plt.title("Gender Representation")
plt.xlabel("Gender")
plt.ylabel("Number of Customers")
plt.tight_layout()
plt.show()
# -----
# 7. Analyze Credit Risk by Gender
# -----
print("\n===== 7. CREDIT RISK BY GENDER =====")
gender_risk = pd.crosstab(
    df["Sex"],
    df["Risk"],
    normalize="index"
) * 100
print(gender_risk)

```

```

plt.figure(figsize=(8, 5))
sns.countplot(
    data=df,
    x="Sex",
    hue="Risk",
    palette="Set1"
)
plt.title("Credit Risk by Gender")
plt.xlabel("Gender")
plt.ylabel("Number of Customers")
plt.tight_layout()
plt.show()
# -----
# 8. Analyze Age Groups
# -----
print("\n===== 8. AGE GROUPS =====")
df["AgeGroup"] = pd.cut(
    df["Age"],
    bins=[18, 25, 35, 50, 65, 100],
    labels=[
        "18-25",
        "26-35",
        "36-50",
        "51-65",
        "66+"
    ],
    include_lowest=True
)
print(df["AgeGroup"].value_counts().sort_index())
plt.figure(figsize=(8, 5))
sns.countplot(
    data=df,
    x="AgeGroup",
    hue="AgeGroup",
    order=["18-25", "26-35", "36-50", "51-65", "66+"],
    palette="viridis"
)
plt.title("Age Group Representation")
plt.xlabel("Age Group")
plt.ylabel("Number of Customers")
plt.tight_layout()
plt.show()
# -----
# 9. Analyze Credit Risk by Age Group
# -----
print("\n===== 9. CREDIT RISK BY AGE GROUP =====")

```

```

age_risk = pd.crosstab(
    df["AgeGroup"],
    df["Risk"],
    normalize="index"
) * 100
print(age_risk)
plt.figure(figsize=(9, 5))
sns.countplot(
    data=df,
    x="AgeGroup",
    hue="Risk",
    order=["18-25", "26-35", "36-50", "51-65", "66+"],
    palette="Set1"
)
plt.title("Credit Risk by Age Group")
plt.xlabel("Age Group")
plt.ylabel("Number of Customers")
plt.tight_layout()
plt.show()
# -----
# 10. Analyze Housing Category Representation
# -----
print("\n===== 10. HOUSING CATEGORY REPRESENTATION =====")
print(df["Housing"].value_counts())
plt.figure(figsize=(8, 5))
sns.countplot(
    data=df,
    x="Housing",
    hue="Housing",
    palette="coolwarm"
)
plt.title("Housing Category Representation")
plt.xlabel("Housing Category")
plt.ylabel("Number of Customers")
plt.tight_layout()
plt.show()
# -----
# 11. Analyze Credit Risk by Housing Category
# -----
print("\n===== 11. CREDIT RISK BY HOUSING CATEGORY =====")
housing_risk = pd.crosstab(
    df["Housing"],
    df["Risk"],
    normalize="index"
) * 100
print(housing_risk)

```

```
plt.figure(figsize=(9, 5))
sns.countplot(
    data=df,
    x="Housing",
    hue="Risk",
    palette="Set1"
)
plt.title("Credit Risk by Housing Category")
plt.xlabel("Housing Category")
plt.ylabel("Number of Customers")
plt.tight_layout()
plt.show()
Output:-
```

```
===== 1. DATASET INFORMATION =====
First 5 rows:
   Unnamed: 0  Age  Sex  Job  Housing  Saving accounts  Checking account  \
0           0    67  male   2    own             NaN             little
1           1    22  female 2    own             little           moderate
2           2    49  male   1    own             little             NaN
3           3    45  male   2    free             little             little
4           4    53  male   2    free             little             little

   Credit amount  Duration  Purpose  Risk
0           1169         6  radio/TV  good
1           5951        48  radio/TV  bad
2           2096        12  education  good
3           7882        42  furniture/equipment  good
4           4870        24         car  bad

Dataset Shape:
(1000, 11)

Column Names:
['Unnamed: 0', 'Age', 'Sex', 'Job', 'Housing', 'Saving accounts', 'Checking account', 'Credit amount', 'Duration', 'Purpose', 'Risk']
```

```
Dataset Information:
<class 'pandas.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 11 columns):
#   Column              Non-Null Count  Dtype
---  ---
0   Unnamed: 0          1000 non-null  int64
1   Age                 1000 non-null  int64
2   Sex                 1000 non-null  str
3   Job                 1000 non-null  int64
4   Housing              1000 non-null  str
5   Saving accounts      817 non-null   str
6   Checking account     606 non-null   str
7   Credit amount        1000 non-null  int64
8   Duration             1000 non-null  int64
9   Purpose              1000 non-null  str
10  Risk                 1000 non-null  str
dtypes: int64(5), str(6)
memory usage: 86.1 KB
```

Statistical Summary:

	Unnamed: 0	Age	Sex	Job	Housing	Saving accounts \
count	1000.000000	1000.000000	1000	1000.000000	1000	817
unique	NaN	NaN	2	NaN	3	4
top	NaN	NaN	male	NaN	own	little
freq	NaN	NaN	690	NaN	713	603
mean	499.500000	35.546000	NaN	1.904000	NaN	NaN
std	288.819436	11.375469	NaN	0.653614	NaN	NaN
min	0.000000	19.000000	NaN	0.000000	NaN	NaN
25%	249.750000	27.000000	NaN	2.000000	NaN	NaN
50%	499.500000	33.000000	NaN	2.000000	NaN	NaN
75%	749.250000	42.000000	NaN	2.000000	NaN	NaN
max	999.000000	75.000000	NaN	3.000000	NaN	NaN

===== 2. DATA QUALITY =====

Missing Values:

Unnamed: 0	0
Age	0
Sex	0
Job	0
Housing	0
Saving accounts	183
Checking account	394
Credit amount	0
Duration	0
Purpose	0
Risk	0
dtype: int64	

Duplicate Rows:

0

Data Types:

Unnamed: 0	int64
Age	int64
Sex	str
Job	int64
Housing	str
Saving accounts	str
Checking account	str
Credit amount	int64
Duration	int64
Purpose	str
Risk	str
dtype: object	

===== 3. TARGET VARIABLE =====

Target Variable: Risk

Target Values:

Risk

good 700

bad 300

Name: count, dtype: int64

===== 4. CLASS DISTRIBUTION =====

Risk

good 700

bad 300

Name: count, dtype: int64

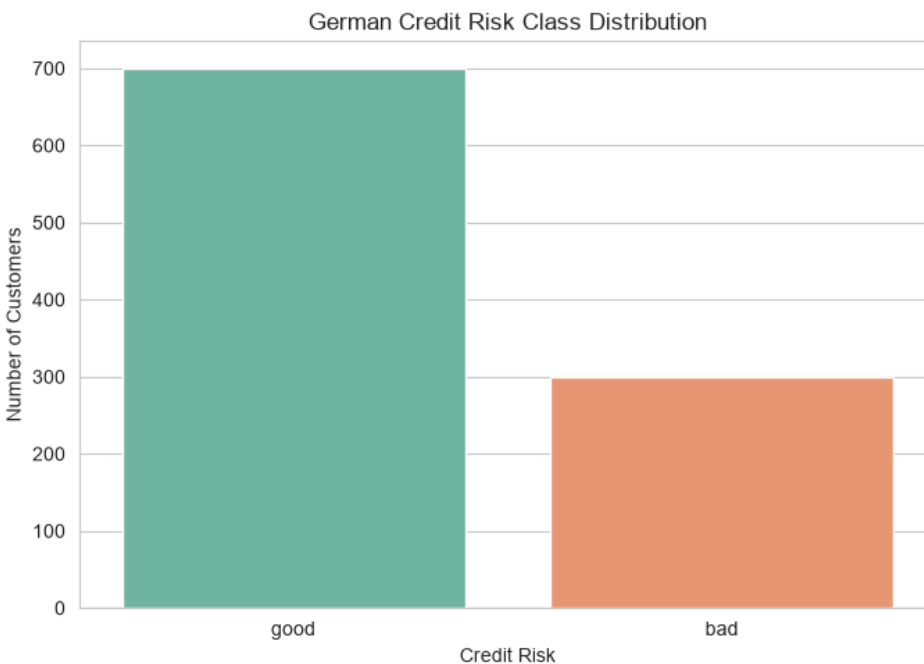
Percentage Distribution:

Risk

good 70.0

bad 30.0

Name: proportion, dtype: float64



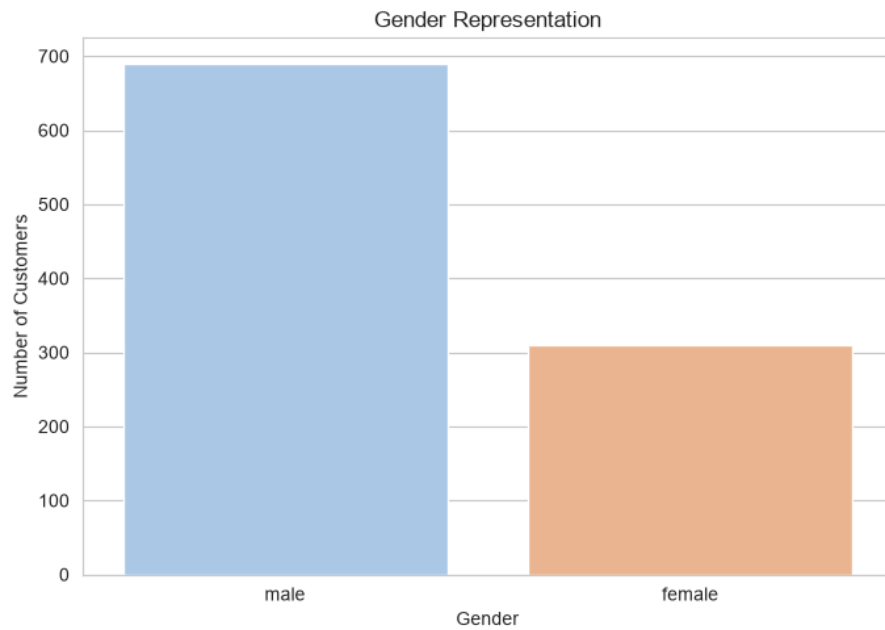
===== 6. GENDER REPRESENTATION =====

Sex

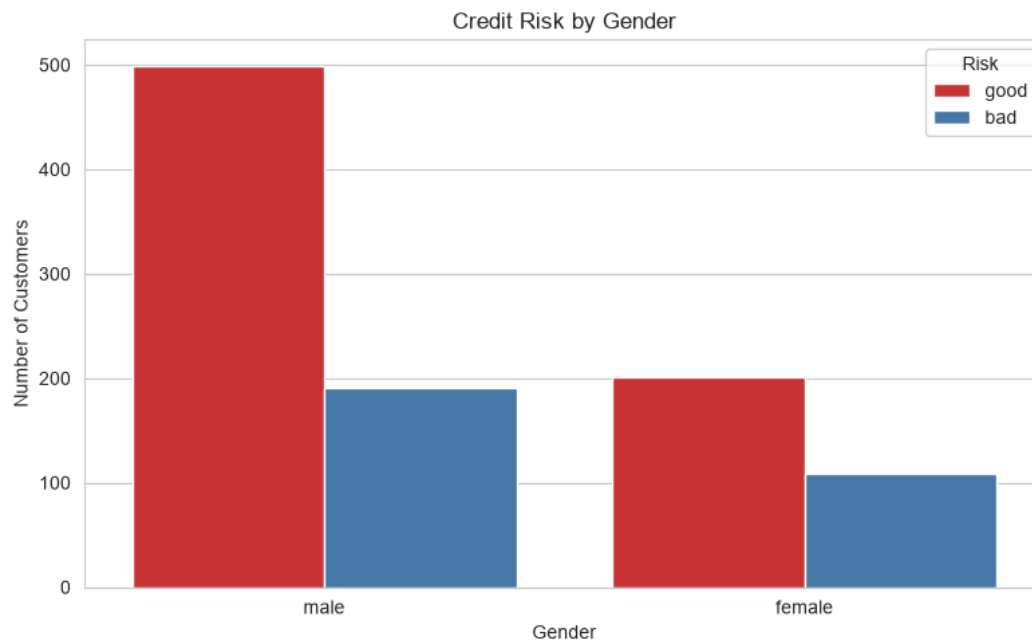
male 690

female 310

Name: count, dtype: int64



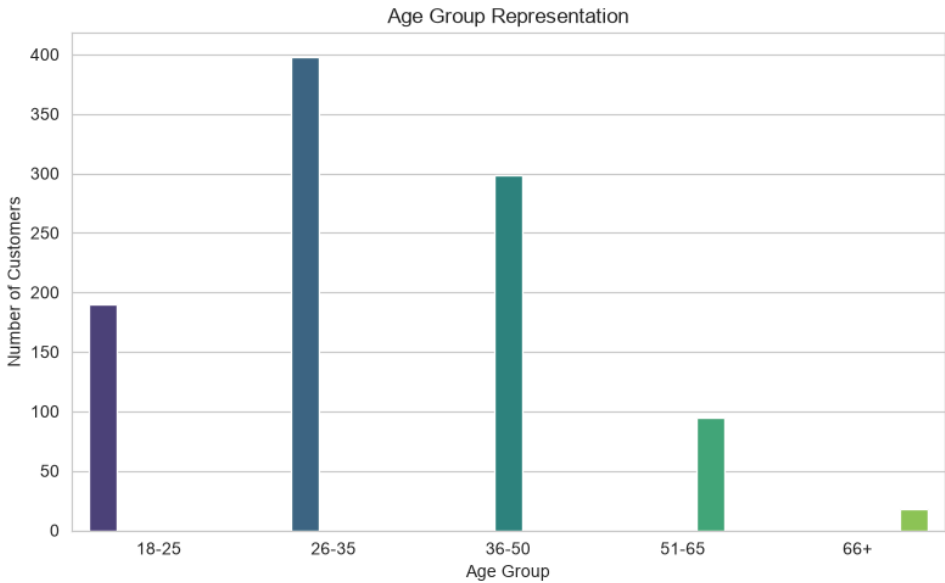
```
===== 7. CREDIT RISK BY GENDER =====  
Risk      bad      good  
Sex  
female  35.161290  64.838710  
male    27.681159  72.318841
```



```

===== 8. AGE GROUPS =====
AgeGroup
18-25    190
26-35    398
36-50    299
51-65     95
66+       18
Name: count, dtype: int64

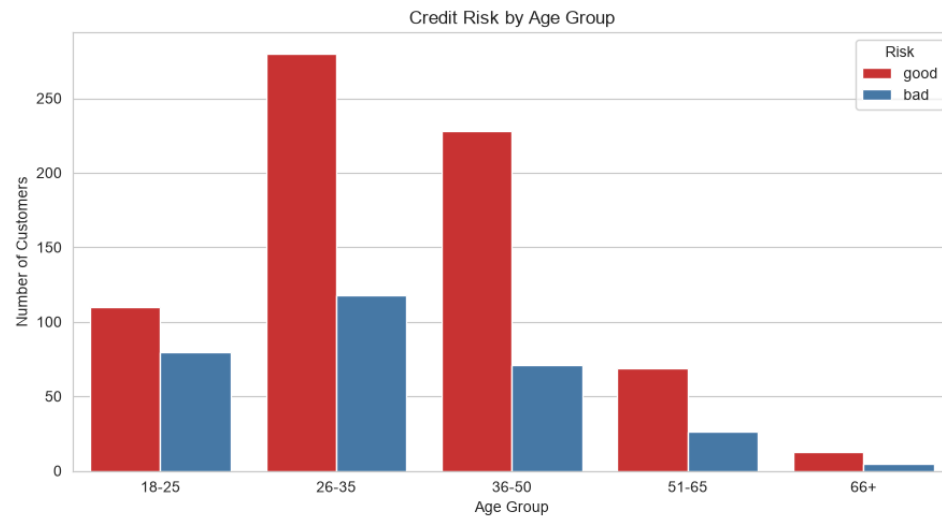
```



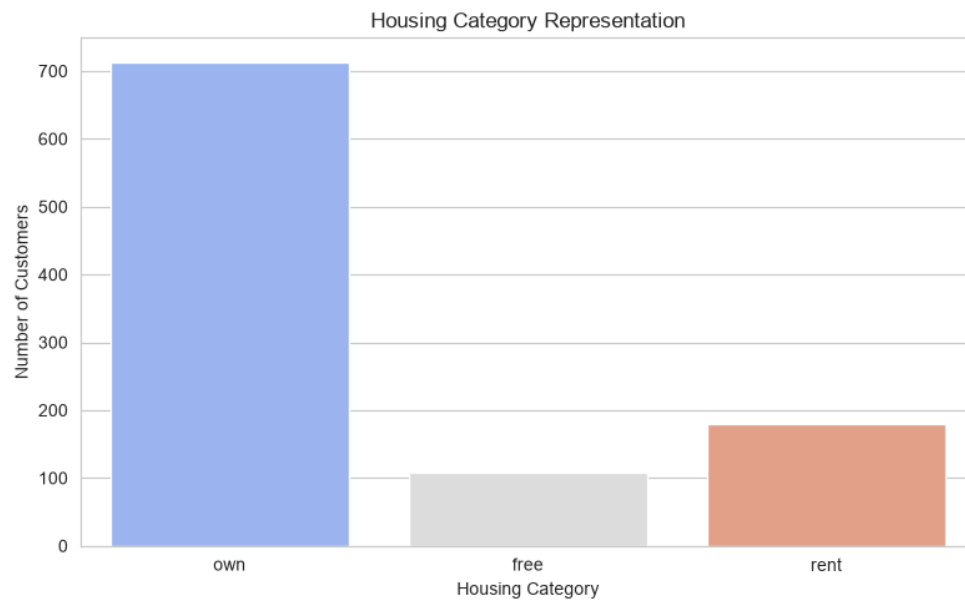
```

===== 9. CREDIT RISK BY AGE GROUP =====
Risk      bad      good
AgeGroup
18-25    42.105263  57.894737
26-35    29.648241  70.351759
36-50    23.745819  76.254181
51-65    27.368421  72.631579
66+      27.777778  72.222222

```



```
===== 10. HOUSING CATEGORY REPRESENTATION =====
Housing
own      713
rent     179
free     108
Name: count, dtype: int64
```



```
===== 11. CREDIT RISK BY HOUSING CATEGORY =====
Risk      bad      good
Housing
free    40.740741  59.259259
own     26.086957  73.913043
rent    39.106145  60.893855
```



German Credit Risk Dataset

12. Identify Possible Bias

The German Credit Risk dataset may contain bias related to gender, age, housing status and credit-risk categories. Some demographic groups may have more records than others. The distribution of good and bad credit-risk cases may also be unequal. Housing and other socioeconomic features may indirectly represent a person's financial or social background. These patterns can result in biased predictions if they are learned directly by a machine learning model.

13. Impact on Machine Learning

Bias in the German Credit dataset can cause a machine learning model to make unequal or inaccurate predictions for different groups. Class imbalance may cause the model to favor the majority credit-risk category. The model may also learn historical socioeconomic patterns associated with gender, age or housing. Consequently, a model may have high overall accuracy while still performing poorly for certain demographic groups.

14. Suggested Solutions

The following solutions can be used to reduce bias:

- Check and handle missing values properly.

- Balance the target classes using oversampling, undersampling or SMOTE.

- Use stratified sampling when dividing the dataset into training and testing sets.

- Evaluate the model using precision, recall, F1-score and ROC-AUC.

- Compare model performance across gender and age groups.

- Perform fairness analysis to identify unequal predictions.

- Carefully evaluate sensitive attributes and features that may act as proxies for them.

- Monitor the model regularly after deployment to detect unfair outcomes.

15. Final Conclusion

The German Credit Risk dataset contains demographic and socioeconomic information that may introduce bias into machine learning predictions. Differences in gender, age, housing categories and credit-risk classes can affect model performance and fairness. Therefore, proper preprocessing, class balancing, suitable evaluation metrics and fairness analysis are necessary to develop a reliable and responsible credit-risk prediction model.